



CENTRO UNIVERSITARIO DE
CIENCIAS EXACTAS E INGENIERÍAS

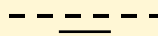


SQLite

Interacción Humano Computadora



Practica #10



Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Carrera: ICOM

Profesor: Jovan Ricardo Zepeda Gómez

Fecha: 04/10/2026

Índice

Índice	1
Tabla de ilustraciones.....	1
Reporte	1
SQLite.....	2
Clases.....	2
Vista.....	9
Bibliografía	11

Tabla de ilustraciones

Ilustración 1. Main Activity class. Fuente: proyecto dbmaster.	2
Ilustración 2. Clase lista. Fuente: proyecto dbmaster.	3
Ilustración 3. Métodos "onCreate, onUpgrede, guardar" de la clase de operaciones de la base de datos. Fuente: proyecto dbmaster.	4
Ilustración 4. Métodos "actualizar, buscar_reg" de la clase de operaciones de la base de datos. Fuente: proyecto dbmaster.	5
Ilustración 5. Métodos "eliminar, llenar_lv" de la clase de operaciones de la base de datos. Fuente: proyecto dbmaster.	6
Ilustración 6. Métodos "controllers" de la vista "buscar_eliminar_actualizar". Fuente: proyecto dbmaster.	7
Ilustración 7. Métodos "controllers" de la vista "bucar_eliminar_actualizar". Fuente: proyecto dbmaster.	8
Ilustración 8. Vista principal. Fuente: proyecto dbmaster.	9
Ilustración 9. Vista lista de resultados. Fuente proyecto dbmaster.	10
Ilustración 10. Vista CRUD base de datos. Fuente proyecto dbmaster.	11

Reporte

Para esta actividad se implemento una pequeña base de datos con el motor SQLite, además del uso de más contenedores vistos anteriormente.

SQLite

SQLite

Es una biblioteca que implementa el motor de base de datos SQL embebido, de configuración cero. El código es de dominio público y cualquier persona puede usarlo libremente para fines comerciales o de uso privado. Aunque es compacto es la base de datos más extendida del mundo y se encuentra en la mayoría de las aplicaciones que utilizamos hoy en día.

Clases

```
1 package com.example.dbmaster;
2
3 > import ...
4
5
6
7
8
9
10
11
12 <> public class MainActivity extends AppCompatActivity {
13     EditText Enombre, Eapellido;
14     Button guardar, buscar;
15
16     @Override
17     protected void onCreate(Bundle savedInstanceState) {
18         super.onCreate(savedInstanceState);
19         setContentView(R.layout.activity_main);
20         Enombre=(EditText)findViewById(R.id.nombre);
21         Eapellido=(EditText)findViewById(R.id.apellido);
22         guardar=(Button)findViewById(R.id.guardar);
23         buscar=(Button)findViewById(R.id.buscar);
24
25         guardar.setOnClickListener(new View.OnClickListener() {
26             @Override
27             public void onClick(View v) {
28
29                 DB db= new DB(getApplicationContext(), name: null, factory: null, version: 1);
30                 String nombre = Enombre.getText().toString();
31                 String apellido = Eapellido.getText().toString();
32                 String mensaje =db.guardar(nombre, apellido);
33                 Toast.makeText(getApplicationContext(),mensaje,Toast.LENGTH_SHORT).show();
34             }
35         });
36
37         buscar.setOnClickListener(new View.OnClickListener() {
38             @Override
39             public void onClick(View v) {
40                 Intent intent = new Intent( packageContext: MainActivity.this, Buscar_eliminar_actualizar.class);
41                 startActivity(intent);
42             }
43         });
44     }
45 }
46 }
```

Ilustración 1. Main Activity class. Fuente: proyecto dbmaster.

La clase principal se encarga de mostrar al usuario las siguientes entradas:

- Nombre (Edit text).

- Apellido (Edit text).
- Guardar (Botón).
- Buscar (Botón).

El primer método “onClick” está destinado al botón guardar, y al presionar guarda lo que existe en los campos “Nombre, Apellido” y los manda a la base de datos. Además de que notifica al usuario de que la operación se realizó correctamente.

El segundo método “onClick” se encarga de dirigir a la siguiente vista donde se encuentran todas las operaciones que se pueden realizar con la base de datos.

```
1      package com.example.dbmaster;
2
3
4
5      > import ...
13
14  <> public class Lista extends AppCompatActivity {
15      ListView lv ;
16      ArrayList<String> lista;
17      ArrayAdapter adaptador;
18      @Override
19  @ protected void onCreate(Bundle savedInstanceState) {
20      super.onCreate(savedInstanceState);
21      setContentView(R.layout.activity_lista);
22      lv = (ListView)findViewById(R.id.lista);
23      DB db = new DB(getApplicationContext(), name: null, factory: null, version: 1);
24      lista = db.llenar_lv();
25      adaptador = new ArrayAdapter(context: this, android.R.layout.simple_list_item_1, lista);
26      lv.setAdapter(adaptador);
27  }
28
29  }
```

Ilustración 2. Clase lista. Fuente: proyecto dbmaster.

Esta clase se encarga de descargar todos los elementos dentro de la base de datos “nombre, apellido” y mostrarlos dentro del contenedor “ListView”, lo hace a través del método “llenar_lv” de la clase “DB”.

```
1 package com.example.dbmaster;
2
3 > import ...
13
14 /**
15  * Created by alexander on 18/09/15.
16  */
17 public class DB extends SQLiteOpenHelper {
18     public DB(Context context, String name, SQLiteDatabase.CursorFactory factory, int version) {
19         super(context, name: "Prueba", factory, version: 1);
20     }
21
22     @Override
23     public void onCreate(SQLiteDatabase db) {
24         db.execSQL("create table datos(nombre text, apellido text)");
25     }
26
27     @Override
28     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
29
30     }
31     public String guardar(String nombre, String apellido){
32         String mensaje="";
33         SQLiteDatabase database = this.getWritableDatabase();
34         ContentValues contenedor = new ContentValues();
35         contenedor.put("nombre", nombre);
36         contenedor.put("apellido", apellido);
37         try {
38             database.insertOrThrow( table: "datos", nullColumnHack: null, contenedor);
39             mensaje="Ingresado Correctamente";
40         }catch (SQLException e){
41             mensaje="No Ingresado";
42         }
43         database.close();
44         return mensaje;
45     }
```

Ilustración 3. Métodos “onCreate, onUpgrede, guardar” de la clase de operaciones de la base de datos. Fuente: proyecto dbmaster.

Dentro de la clase “db” se encuentran todos los métodos que define el comportamiento de la base de datos, empezando por:

- Constructor: S encarga de inicializar todos los parámetros que necesita la base de datos.
- “onCreate”: Método necesario de la clase heredada y se utiliza para modificar el comportamiento al crear la clase. En este caso, utilizando lenguaje SQL crea una base de datos nueva al inicial la clase.
- “onUpgrade”: Segundo método necesario de la clase padre que se utiliza para verificar y modificar el comportamiento de la base de datos en cada actualización externa que se vaya a realizar.

- “guardar”: Guarda los valores obtenidos “nombre, apellido” dentro de un contenedor y al final dentro de un bloque “try-catch” guarda la base de datos. Y cierra la comunicación de la base de datos.
- “actualizar”: Obtiene tres parámetros, “buscar” que es el nombre a encontrar dentro de la base de datos y los nuevos nombres por los que se van a actualizar. Primero realiza una búsqueda, la instrucción arroja una excepción si no lo encontró o falló al realizar la búsqueda, si no hubo nada entonces se despliega el mensaje de que se realizó correctamente.
- “buscar_reg”: Se encarga de mostrar en base a un nombre como referencia el registro e imprimirlo en pantalla si existe algún registro dentro de la base de datos. Al final notifica al usuario si no se encontró el registro.
- “eliminar”: Elimina con el parámetro “buscar” el registro que coincida con la referencia y al final notifica al usuario.
- “llevar_lv”: Llena el contenedor de lista “ListView” obteniendo todos los registros existentes dentro de la base de datos y transformándolos en una lista de Strings.

```
46 public String actualizar(String Buscar,String Nombre,String Apellido){
47     String Mensaje="";
48     SQLiteDatabase database = this.getWritableDatabase();
49     ContentValues contenedor = new ContentValues();
50     contenedor.put("nombre",Nombre);
51     contenedor.put("apellido",Apellido);
52     int cantidad = database.update( table: "datos", contenedor, whereClause: "nombre='"+ Buscar + "'", whereArgs: null);
53     if(cantidad!=0){
54         Mensaje="Actualizado Correctamente";
55     }else{
56         Mensaje="No Actualizado";
57     }
58     database.close();
59     return Mensaje;
60 }
61
62 public String[] buscar_reg(String buscar){
63     String[] datos= new String[3];
64     SQLiteDatabase database = this.getWritableDatabase();
65     String q = "SELECT * FROM datos WHERE nombre='"+buscar+"'";
66     Cursor registros = database.rawQuery(q, selectionArgs: null);
67     if(registros.moveToFirst()){
68         for(int i = 0 ; i<2;i++){
69             datos[i]= registros.getString(i);
70         }
71         datos[2]= "Encontrado";
72     }else{
73         datos[2]= "No se encontro a "+buscar;
74     }
75     database.close();
76     return datos;
77 }
78
79 }
```

Ilustración 4. Métodos “actualizar, buscar_reg” de la clase de operaciones de la base de datos. Fuente: proyecto dbmaster.

SQLite

```
80     public String eliminar(String Nombre){
81         String mensaje = "";
82         SQLiteDatabase database = this.getWritableDatabase();
83         int cantidad = database.delete( table: "datos", whereClause: "nombre='" + Nombre + "'", whereArgs: null);
84         if (cantidad !=0){
85             mensaje="Eliminado Correctamente";
86         }
87     }
88     else{
89         mensaje = "No existe";
90     }
91     database.close();
92     return mensaje;
93 }
94 public ArrayList llenar_lv(){
95     ArrayList<String> lista = new ArrayList<>();
96     SQLiteDatabase database = this.getWritableDatabase();
97     String q = "SELECT * FROM datos";
98     Cursor registros = database.rawQuery(q, selectionArgs: null);
99     if(registros.moveToFirst()){
100         do{
101             lista.add(registros.getString(1));
102         }while(registros.moveToNext());
103     }
104     return lista;
105 }
106 }
107
108 }
```

Ilustración 5. Métodos "eliminar, llenar_lv" de la clase de operaciones de la base de datos. Fuente: proyecto dbmaster.

```

1 package com.example.dbmaster;
2 import ...
14
15 public class Buscar_eliminar_actualizar extends AppCompatActivity {
16     EditText Ebuscar, nnombre, napellido;
17     TextView nombre, apellido;
18     Button Bbuscar, Beliminar, Bactualizar, Blista;
19     @Override
20     protected void onCreate(Bundle savedInstanceState) {
21         super.onCreate(savedInstanceState);
22         setContentView(R.layout.activity_buscar_eliminar_actualizar);
23         nombre=(TextView)findViewById(R.id.nombre);
24         apellido=(TextView)findViewById(R.id.apellido);
25         Ebuscar=(EditText)findViewById(R.id.Ebuscar);
26         Bbuscar=(Button)findViewById(R.id.Bbuscar);
27         Beliminar=(Button)findViewById(R.id.Beliminar);
28         nnombre=(EditText)findViewById(R.id.NNombre);
29         napellido=(EditText)findViewById(R.id.NApellido);
30         Bactualizar=(Button)findViewById(R.id.BActualizar);
31         Blista=(Button)findViewById(R.id.blista);
32         Bbuscar.setOnClickListener(new View.OnClickListener() {
33             @Override
34             public void onClick(View v) {
35                 DB db = new DB(getApplicationContext(), name: null, factory: null, version: 1);
36                 String buscar = Ebuscar.getText().toString();
37                 String[] datos;
38                 datos=db.buscar_reg(buscar);
39                 nombre.setText(datos[0]);
40                 apellido.setText(datos[1]);
41                 Toast.makeText(getApplicationContext(),datos[2],Toast.LENGTH_SHORT).show();
42             }
43         });
44         Beliminar.setOnClickListener(new View.OnClickListener() {
45             @Override
46             public void onClick(View v) {
47                 DB db = new DB(getApplicationContext(), name: null, factory: null, version: 1);
48                 String Nombre = nombre.getText().toString();
49                 String mensaje =db.eliminar(Nombre);
50                 Toast.makeText(getApplicationContext(),mensaje,Toast.LENGTH_SHORT).show();
51             }
52         });

```

Ilustración 6. Métodos “controllers” de la vista “buscar_eliminar_actualizar”. Fuente: proyecto dbmaster.

La clase “buscar-eliminar-actualizar” se encarga de controlar la vista que interactúa con las operaciones principales de la base de datos. Y es quien obtiene los parámetros necesarios para ejecutar los métodos de la base de datos.

```
53     BActualizar.setOnClickListener(new View.OnClickListener() {
54         @Override
55         public void onClick(View v) {
56             DB db = new DB(getApplicationContext(), name: null, factory: null, version: 1);
57             String Buscar = nombre.getText().toString();
58             String Nombre = nnombre.getText().toString();
59             String Apellido = napellido.getText().toString();
60             String Mensaje = db.actualizar(Buscar, Nombre, Apellido);
61             Toast.makeText(getApplicationContext(), Mensaje, Toast.LENGTH_SHORT).show();
62         }
63     });
64     BLista.setOnClickListener(new View.OnClickListener() {
65         @Override
66         public void onClick(View v) {
67             Intent intento = new Intent(getApplicationContext(), Lista.class);
68             startActivity(intento);
69         }
70     });
71 }
```

Ilustración 7. Métodos “controllers” de la vista “bucar_eliminar_actualizar”. Fuente: proyecto dbmaster.

Vista

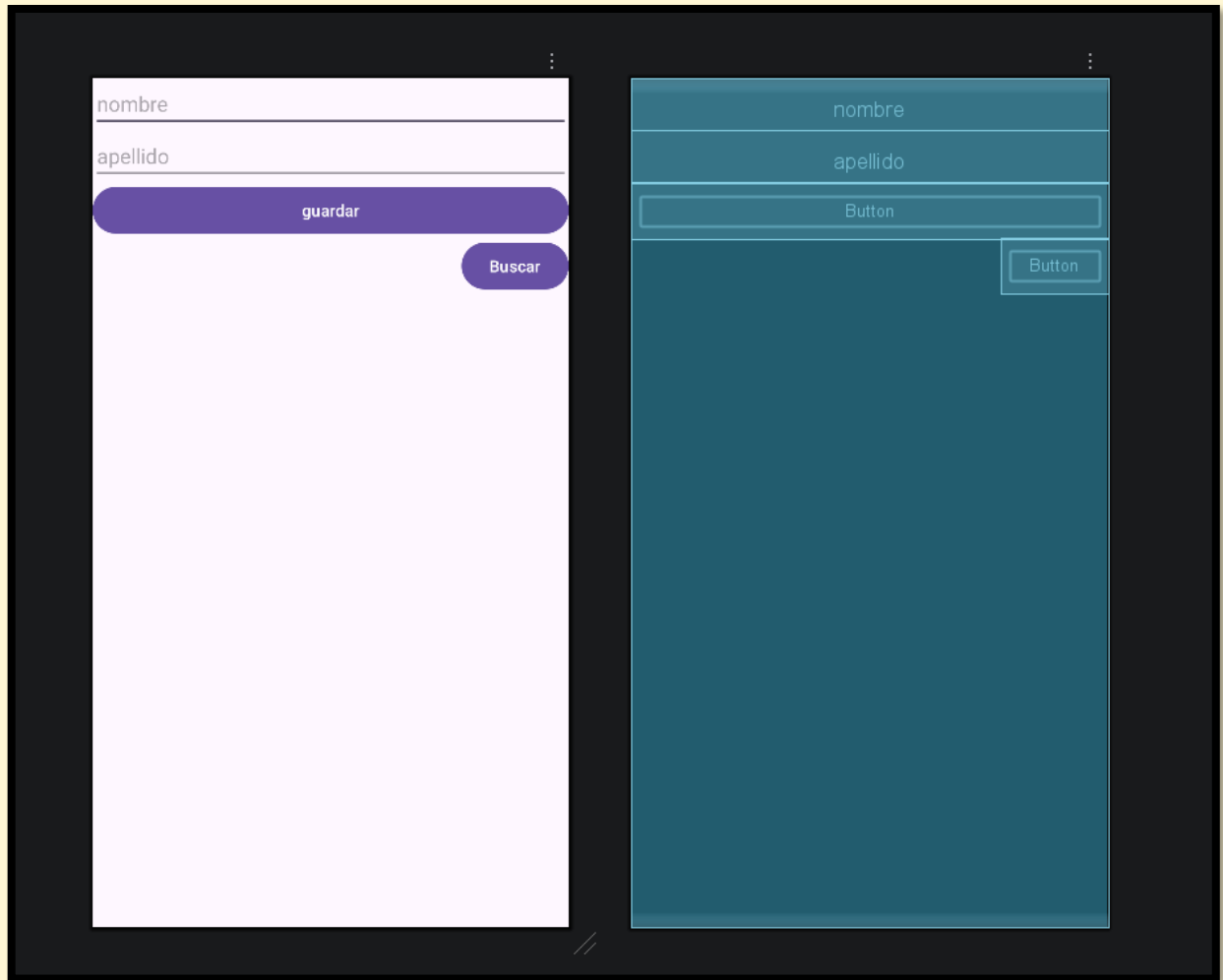


Ilustración 8. Vista principal. Fuente: proyecto dbmaster.

SQLite

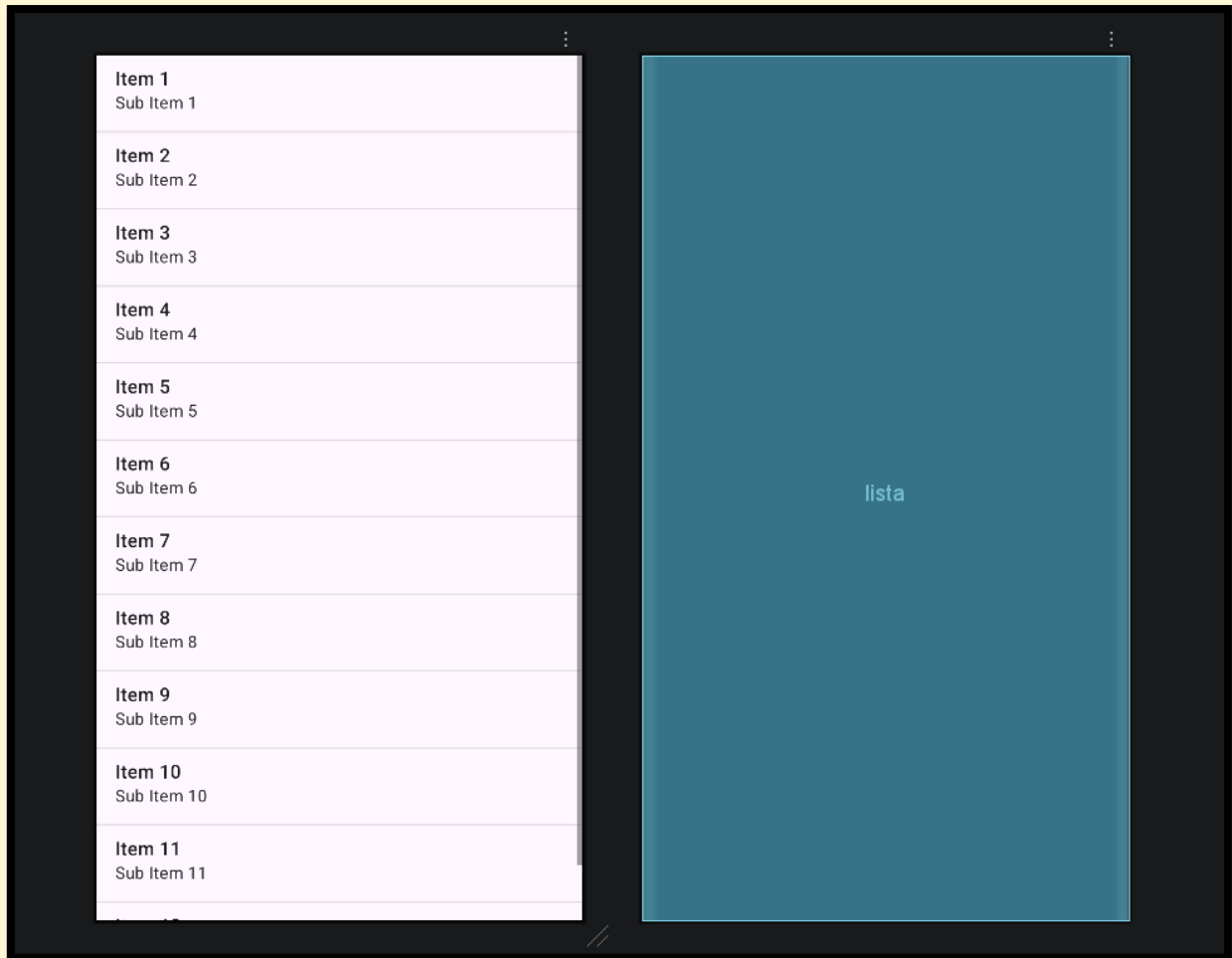


Ilustración 9. Vista lista de resultados. Fuente proyecto dbmaster.

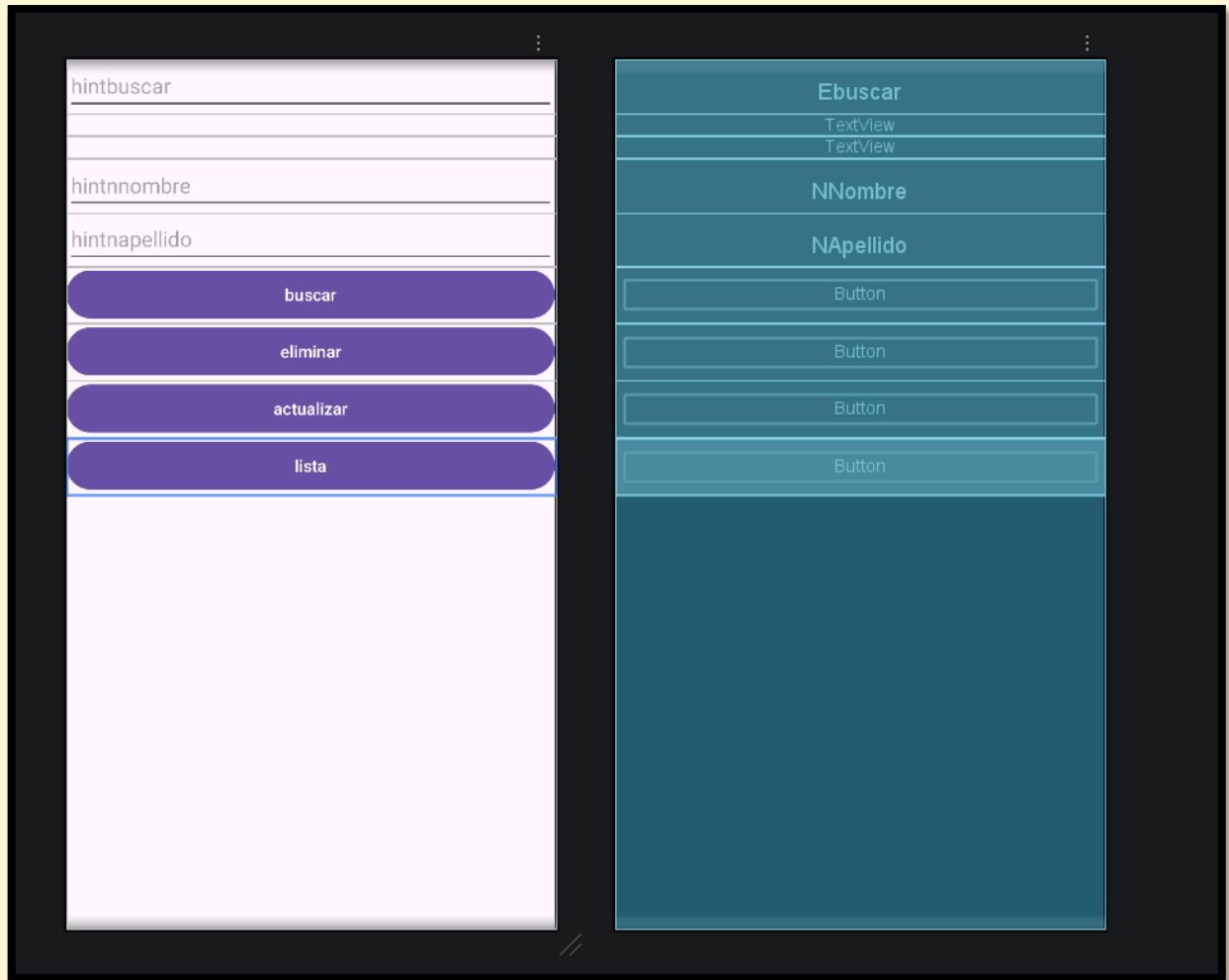


Ilustración 10. Vista CRUD base de datos. Fuente proyecto dbmaster.

Bibliografía

- About SQLite. (s/f). Sqlite.org. Recuperado el 5 de octubre de 2026, de <https://sqlite.org/about.html>
- Android API reference. (s/f). Recuperado el 04 de octubre de 2026, de Android Developers website: <https://developer.android.com/reference>.

